SUNG KYUN KWAN UNIVERSITY(SKKU)

Reinforcement Learning for Taxi Dispatching
: Final Project Report

Team 4.
권도균, 전이루, 정찬희, 황지영

Contents

1. Problem Definition
2. Related Research
3. Proposed Method
4. Q-learning experiment & Result
5. DQN experiment & Result
6. Results Comparison
7. Conclusion

Introduction

Background of vehicle dispatching

- **Traditional approach: Greedy Strategy**

  Dispatch the nearest vehicle.

  May miss the optimal long-term performance.

- **Need to maximize cumulative efficiency, not immediate gain.**

  Strategic positioning based on future demand and vehicle’s situation.

Introduction

Problem Definition

- **Goal:** Minimize vacant time and maximize revenue of a taxi.
- **Why should we solve this problem?**
  - Limitation of greedy system
- **Why Reinforcement Learning?**
  - Dispatching problem: a sequential decision making problem
  - Agent learns optimal policy through continuous interaction with environment.

Related Works

Dietterich (2000): Taxi environment as a benchmark for hierarchical RL

Mnih et al. (2015): DQN extends Q-learning using neural networks

Zhou et al. (2023): robust and multi-agent RL for large-scale dispatching

Rajeh et al. (2024): Clustering-based Multi-Agent RL
for Larger Scale Taxi dispatching

Our project: controlled comparison of Q-learning and DQN in Taxi-v3

Method

MDP Formulation

- $\mathrm{M} = \langle S, A, P, R, \gamma \rangle$
- State Space S
  - Taxi Coordinates (a $N \times N$ grid of locations)
  - Passenger Location (5 states: Destinations + In/Not-in the taxi)
  - Destination (4 designated landmarks $\{R, G, Y, B\}$)
- *Actions* A
  - Movement: South, North, East, West
  - Operational: Pickup | Drop-off passenger
- *Reward* R

$$R(s, a) = \begin{cases} +20 & \text{if delivering a passenger at destination} \\ -10 & \text{if executing Pickup and Drop-off illegally} \\ -1 & \text{other (step-wise penalty)} \end{cases}$$

Q-Learning: Tabular Baseline for Taxi Dispatching

Q-table lookup

Stores Q(s, a) for every discrete state-action pair.

Taxi-v3 has 500 states × 6 actions — small enough for a dense table.

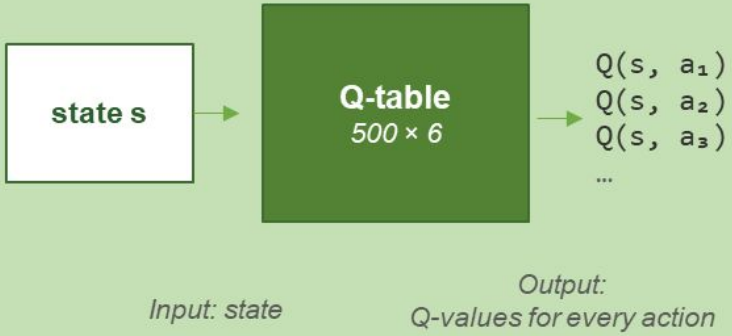
ε-greedy action selection

Balances exploration and exploitation during training.

Temporal-difference update

Bootstraps Q(s, a) from the current reward and the best next-state value.

$$Q(s,a) \leftarrow Q(s,a) + \alpha\,[\, r + \gamma \cdot \max Q(s',a') - Q(s,a)\,]$$



No function approximation — exact Q-values

Small, discrete state space → strong tabular baseline

How Q-Learning Learns

1

Initialize

Q-table set to zeros
(500 × 6 matrix)

2

Act

Pick action via
ε-greedy policy

3

Observe

Execute action,
receive (r, s′)

4

Update

Apply TD update
to Q(s, a)

TD Update $Q(s,a) \leftarrow Q(s,a) + \alpha\,[\, r + \gamma \cdot \max Q(s',a') - Q(s,a)\,]$

Implementation improvements

- **Random tie-breaking** — avoid bias when argmax has ties
- **Best-Q checkpointing** — save table with highest 100-ep MA

Hyperparameters

episodes = 8000, max steps = 200
α = 0.1, γ = 0.99
ε-decay = 0.9992, ε-min = 0.01

Q-Learning Training Result

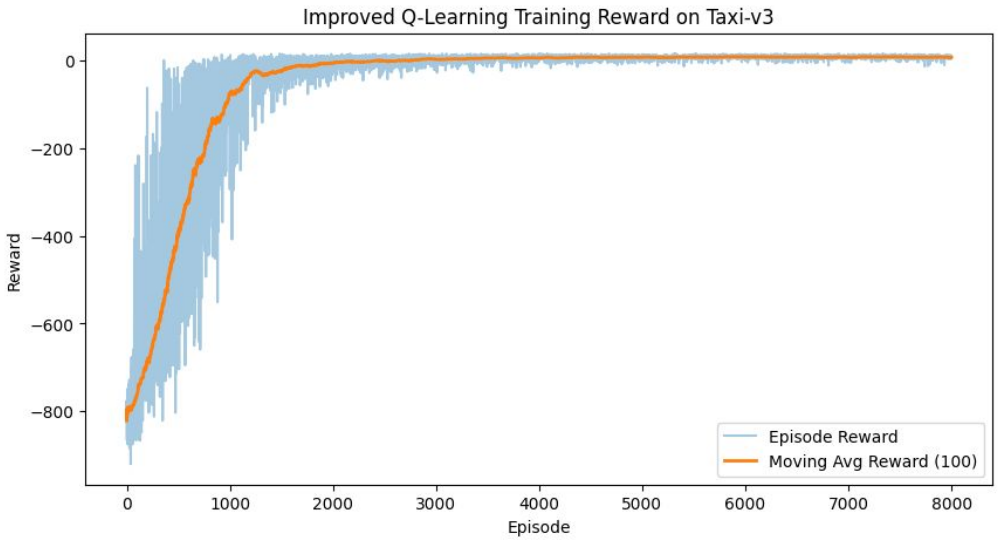


Fig. Improved Q-Learning training reward on Taxi-v3 (8000 episodes)

Interpretation

- **Early instability**

  High ε ⇒ large negative rewards at start
  100-ep MA ≈ −394.82 at episode 500

- **Clear upward trend**

  100-ep MA rises steadily and reaches 7.75 by episode 6000

- **Stable tabular learning**

  Best 100-ep MA = 8.23 — Q-learning converges without function approximation

Q-Learning Evaluation Result

Improved Q-Learning — 200 eval episodes

Metric	Value
Average reward	**7.96**
Average steps	**13.04**
Success ratio (reward ≥ 0)	**1.000**
High-quality eps (reward ≥ 8)	**0.520**

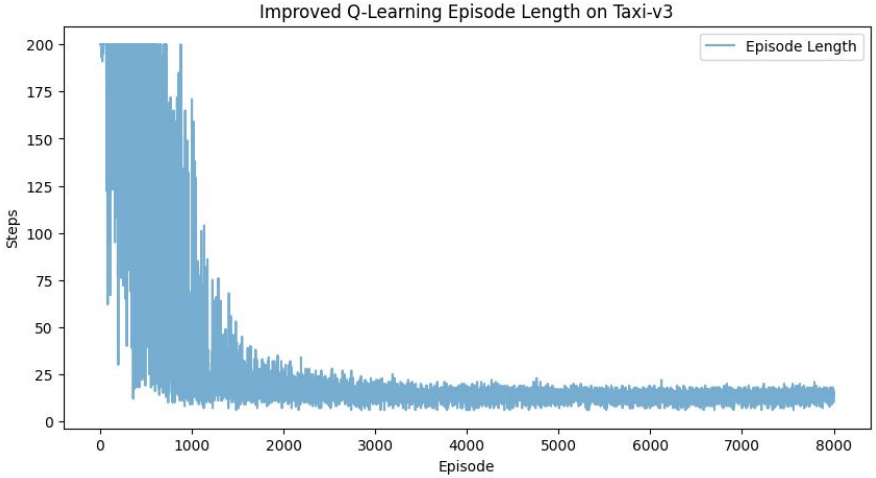


Fig. Episode length during training

Basic Q-learning: reward 7.63 / steps 13.37 → **Improved: 7.96 / 13.04**

Higher reward

Improved model outperforms the basic Q-learning baseline.

Fewer steps

Agent completes pickup-and-dropoff with less wandering.

Reliable policy

All 200 eval episodes end with non-negative reward.

DQN: Neural Q-Function Approximation

Q-table → Neural network

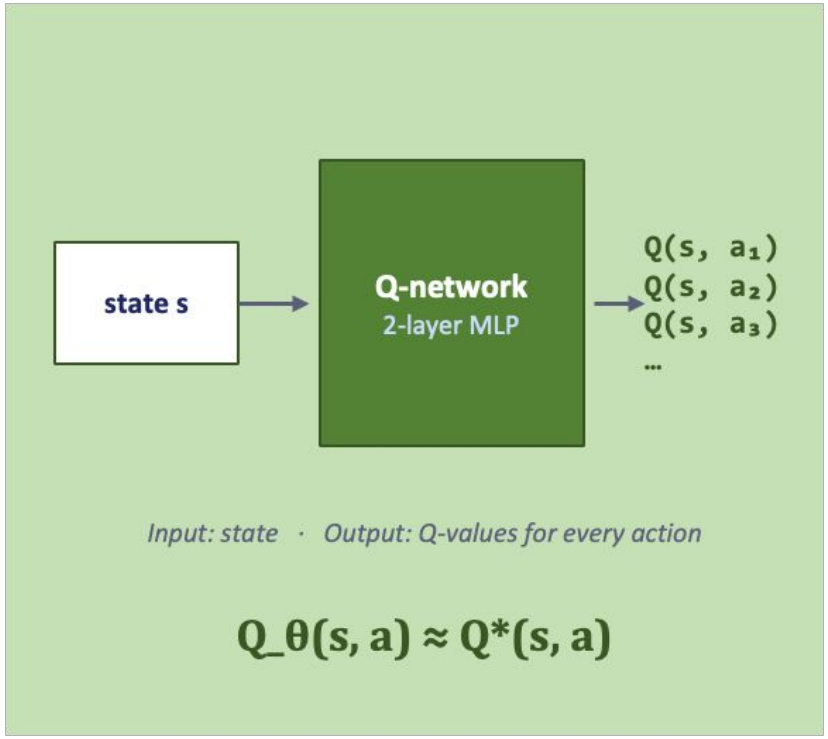
Approximates action-value Q(s, a) with a network instead of storing every (s, a) entry.

ε-greedy action selection

Balances exploration and exploitation during training.

Scalable to larger state spaces

Generalizes across unseen states — no exhaustive table required.



How DQN Learns

1

Store

Save transition (s, a, r, s′) in replay memory

2

Sample

Randomly draw a mini-batch from replay memory

3

Target

$y = r + \gamma \cdot \max_{a'} Q_target(s', a')$

4

Update

Minimize MSE loss via SGD on the Q-network

Loss $L(\theta) = \mathbb{E}[\,(y - Q_\theta(s, a))^2\,]$

Hyperparameters

episodes = 3000, max steps = 200 α (lr) = 1e-3, γ = 0.99 ε-start = 1.0, ε-min = 0.05, ε-decay = 0.995, batch size = 64, buffer size = 10000 target update = 20 hidden = 128×128

Why DQN Converges

Problem: plain neural Q-learning is unstable — correlated samples and a moving target break training.

Experience Replay

Fix

Sample correlation

Randomly sampling past transitions breaks the temporal correlation between consecutive updates.

Also improves data efficiency — the same transition can be reused many times.

Fixed Target Network

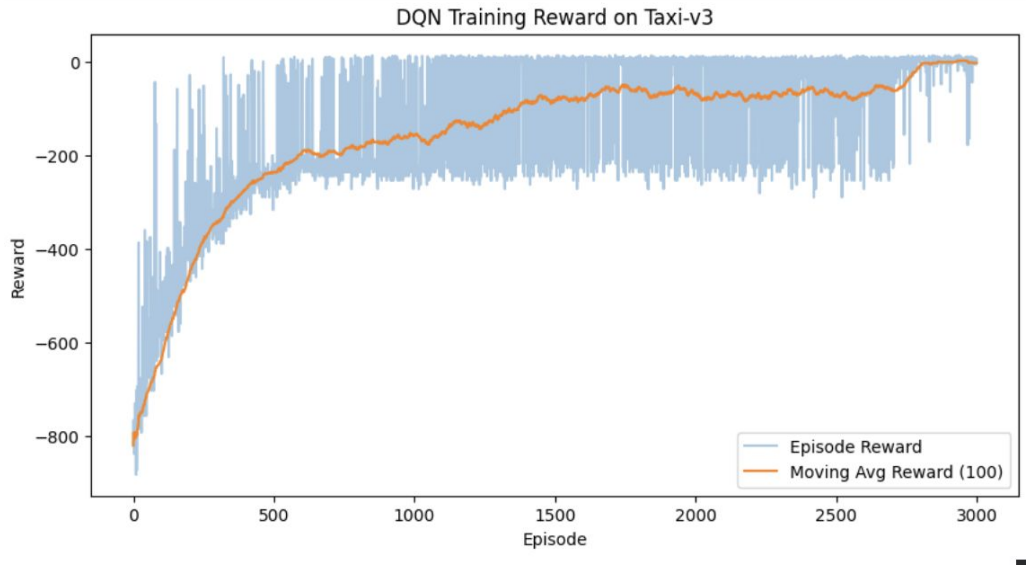
Fix

Non-stationary target

A separate network computes the target and is only copied from the Q-network every N steps.

The target stays fixed briefly, preventing prediction and target from chasing each other.

DQN Training Result



Interpretation

- **Early fluctuation**
  High ε → exploration dominates, reward is noisy.
- **Upward trend**
  Moving-average reward rises as policy improves.
- **Scalability win**
  Handles larger state spaces better than tabular Q-learning – at higher compute cost.

Metric	Value
Average reward	**-6.81**
Average steps	**26.34**

Scalability Experiment

How state-space growth affects each method (custom Env, 1,500 episodes)

Map Size	States	Q-learning reward	DQN reward
10×10	2,000	−646.46	**−400.00**
15×15	4,500	−1,465.74	**−900.00**

Linear Q-table growth

Q-table entries scale linearly with grid cells: Memory cost becomes prohibitive for realistic city grids.

Q-learning collapses

Reward drops sharply to −1,466 as the grid grows. Fixed training budget exposes tabular sample-inefficiency.

DQN degrades gracefully

DQN retains a better reward profile at the largest scale (−900 vs −1,466) via neural function approximation.

Results Comparison

Q-Learning vs DQN — a scale-dependent comparison

Aspect	Q-Learning	DQN
Representation	Tabular Q-table	Neural network (function approx.)
Small env (Taxi-v3 5×5)	7.96	-6.81
Middle env (Custom 10×10)	−646	−400
Large env (Custom 15x15)	−1,466	−900
Best use case	**Small discrete problems**	**Large / scalable spaces**

Q-Learning wins on simplicity
DQN wins on scalability.

Conclusion

Q-Learning is a strong baseline for small dispatching problems,
but DQN becomes essential as the world grows larger.

STRENGTH

Q-Learning is a strong baseline

Reward 7.96 with 100% success on Taxi-v3; tie-breaking + best-model checkpointing stabilize the policy.

LIMITATION

Tabular method fails at scale

Reward collapses to −1,466 at 15×15 grid; Q-table grows linearly to 27K entries under fixed compute.

DIRECTION

DQN scales more gracefully

Function approximation retains −900 at 15×15; a clear path toward realistic city-scale dispatching.

Team Contribution

Name	Task	Description
전이루	Problem Definition, Related Works, MDP Formulation & Set up	○ Define the taxi dispatching problem and research motivation ○ Formulate state, action, reward, and objective as an MDP
권도균	Q-learning Method & Result	○ Explain Q-learning baseline ○ Analyze Q-learning training behavior and evaluation results
황지영	DQN Method & Result	○ Explain DQN architecture ○ Analyze DQN training behavior and evaluation results
정찬희	Scalability Experiment, Comparison, Discussion, Conclusion	○ Compare Q-learning and DQN with scalability analysis ○ Summarize discussion, conclusion ○ Finalize project report and presentation